Azure Databricks Platform Administration and Security

Three sessions: what the platform is and the vocabulary it uses, how it is built and deployed, and how an enterprise deployment is networked.

SESSION 1 · 2H 20M
Introduction to Azure Databricks

SESSION 2 · 1H 30M
Platform Architecture

SESSION 3 · 2H 30M
Network Design and Private Connectivity

What this session covers

01 **What Databricks is, and what it solves**
The lakehouse model, Delta Lake, and where the platform came from.

02 **Databricks architecture**
The pieces of the platform, and the Azure services around them.

03 **Anatomy of the platform**
Account, workspace, compute, notebooks, jobs and the namespace.

04 **Where Databricks sits in the Azure estate**
ADLS Gen2, Entra ID, Key Vault, Monitor and Cost Management.

05 **How work happens, and who does what**
End-to-end flow, platform roles and shared responsibility.

By the end, you will be able to

01 Explain what Azure Databricks is and the data management problem the lakehouse model addresses.

02 Use the platform's core vocabulary correctly: account, workspace, compute, notebook, job, catalog, schema, table and volume.

03 Describe where data physically resides and how Databricks relates to ADLS Gen2, Entra ID, Key Vault, Azure Monitor and Azure Cost Management.

04 Identify the main administrative roles on the platform and who is responsible for what across Microsoft, Databricks and the organisation.

01

What Databricks is, and what it solves

The problem that came before the platform, and the model that answers it.

the problem

the lakehouse

Delta Lake

What is data engineering?

Data engineering is the practice of **building and running the systems** that take raw data from where it is created to a form people can trust and use.

Building

Someone designs and writes the pipelines that move and reshape the data.

Running

They work every day, unattended — and someone is accountable when they do not.

Trust and use

The output is correct, explainable and in a shape people can actually work with.

The data engineering pipeline

SOURCES

Where data is created

- Policy & claims systems
- Finance ledger
- Broker files, CSV, JSON
- Streams and telematics

→

INGESTION

Bringing it in

Batch loads on a schedule, or events arriving continuously. Landed exactly as received, so the original is never lost.

→

DATA ENGINEERING

Making it usable

- Cleaning — bad and duplicate records
- Transforming — conforming and joining
- Aggregating — the shapes people ask for
- Governing — names, grants, lineage

→

CONSUMPTION

What it is all for

- Dashboards and reports
- Ad-hoc analysis
- Features and models
- Regulatory returns

Every Databricks component we meet later sits in one of these four columns.

What is a data warehouse?



The transform happens before the load — so a warehouse always holds clean data, and never holds the raw original.

What is a data lake?



Anything can go in, and nothing has to be thrown away — which is why machine learning work moved here first.

Two systems, two different jobs

DATA WAREHOUSE

Clean tables, reliable reporting

Structured tables, enforced schemas, transactions.

Cannot hold raw files, documents or images.

Expensive to scale; storage and compute tied together.

DATA LAKE

Cheap storage, anything goes

Any format, any volume, on low-cost object storage.

No transactions and no schema enforcement.

Updating or deleting a single record is painful.

Running both is the norm — and running both is where the trouble starts.

What running both costs you

Duplicated data

The same table lives in the lake and again in the warehouse. When a copy fails, the two disagree and nobody knows which is right.

Inconsistent governance

Permissions are granted separately in each system. Answering “who can see this column?” means checking several places.

Pipelines to maintain

The copy jobs between the two become infrastructure of their own — and a source of delay and failure.

> These are not tuning problems. They follow from having two copies of the truth.

Data lake + data warehouse = data lakehouse



Data lake

Cheap, open, accepts anything

+



Data warehouse

Transactions, schema, fast SQL

=

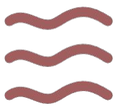


Data lakehouse

One copy of the data, both sets of behaviour

What is a data lakehouse?



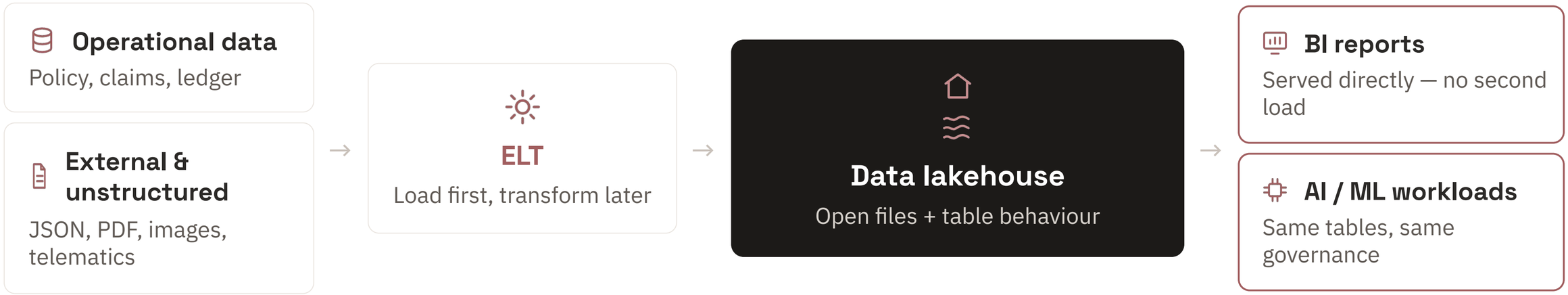


A data lakehouse is a new, **open** data management architecture that combines the flexibility, cost-efficiency and scale of **data lakes** with the data management and ACID transactions of **data warehouses** — enabling BI and machine learning on all data.

— Databricks

> For us that is literal: **ADLS** holds the files, **Delta Lake** gives them table behaviour, **Unity Catalog** governs them, Spark and SQL warehouses do the work.

What the architecture looks like



Compare with the two-tier diagram: **the second copy and the second pipeline are gone.**

The lakehouse: one governed copy

BI & reporting
Dashboards, packs and regulatory returns

Data engineering
Pipelines building the governed tables

Data science
Exploration and feature development

Machine learning
Training, serving and monitoring models

↓ every workload reads the same tables in place — no extract, no copy ↓

GOVERNANCE LAYER

- Permissions set once
- Lineage end to end
- One audit trail
- Discovery and search

ONE STORED COPY

- Delta tables
- Open Parquet files
- On our ADLS Gen2
- Nothing to reconcile

> Warehouse reliability and lake flexibility on one set of files — that combination is what the word lakehouse means.

What is Databricks?

Databricks is **one cloud platform** for everything we do with data — storing it, cleaning it, analysing it and building models — in a single shared workspace that runs **inside our own Azure account**.

Storing it

Delta tables and files on our own ADLS Gen2

Cleaning it

Pipelines and jobs that transform and conform

Analysing it

SQL, dashboards and Power BI on governed tables

Building models

Training, tracking and serving in the same place

Not a service we upload data to. It runs in our subscription, on our storage, under our Entra ID identity.

What is Apache Spark?

Spark is the **engine that does the work** . Data too large for one machine gets split into pieces, each piece is processed on a different machine at the same time, and the answers are combined. You write one query — Spark decides how to spread it out.

YOU WRITE

One query

`SELECT … FROM claims`

→

DRIVER

Plans & splits the job

Breaks the data into chunks and hands them out.

→

EXECUTORS — WORKING IN PARALLEL

Machine 1	Machine 2	Machine 3	Machine 4
Jan–Mar	Apr–Jun	Jul–Sep	Oct–Dec

Partial answers come back and are combined into one result.

A full-portfolio valuation finishes in minutes because twenty machines each do a twentieth of it.

How Spark and Databricks relate

APACHE SPARK ALONE

An open-source engine

Free to download — and yours to install, size, tune, patch and secure.

THE CONNECTION

Same people

Databricks was founded by the team who created Spark at UC Berkeley.

DATABRICKS — THE PLATFORM STACKED ON IT

Workspace, jobs & admin	Where people write code and it gets scheduled and governed
Unity Catalog	Who may see which table, and the lineage of how it was built
Delta Lake	Transactions, schema and history on open files
Photon	A faster query engine underneath Spark

↑ everything above is what Databricks adds ↑

Apache Spark	The engine at the base of the stack — still doing the actual processing

02

Databricks architecture

The pieces of the platform, and the Azure services that sit around them.

the components

what Azure provides

The pieces of Databricks, and what each one does

Clusters	Machines that run notebooks and jobs
SQL warehouses	Compute tuned for queries and dashboards
Delta Lake	The table format: reliable updates and history
Lakeflow pipelines	Declarative pipelines that build the layers
Workflows	Schedules the work and handles retries
Unity Catalog	Decides who may see which table

AT THE CENTRE

Apache Spark

Every component around it runs on Spark

Photon	Faster query engine underneath Spark
Workspace & notebooks	Where people write SQL, Python and R
MLflow	Tracks experiments and deploys models
Databricks IQ	Understands our data; powers search and assist
Admin controls	Users, groups, budgets and workspace settings
Marketplace & sharing	Publishes data to partners without copying it

What Azure provides around it

Microsoft Entra ID
Sign in with your company account — no separate users

Event Hubs
Streaming events into the platform

Unified billing
Databricks usage lands on the same Azure invoice

Key Vault
Secrets and keys, never in notebooks

PREVIOUS SLIDE

Databricks on Spark

Clusters, Delta Lake, Unity Catalog, workflows

ADLS Gen2 storage
Where our tables and files physically live

Azure DevOps
Git repos and deployment pipelines

Azure VMs
The machines clusters and warehouses run on

Azure Monitor
Logs, metrics and alerts for what ran

Databricks architecture: two planes

WHO

Users & applications

Browser, BI tools, APIs.

→

Control plane

RUN BY DATABRICKS

- Web interface & notebooks
- Compute orchestration — starts and stops machines
- Unity Catalog — permissions and lineage
- Saved queries, code and job definitions

↔

Compute plane

WHERE WORK ACTUALLY RUNS

Classic

Clusters on VMs in our own Azure subscription.

Serverless

Databricks supplies the machines; ready in seconds.

↔

OURS

Cloud storage

Every table and file stays in our ADLS account.

Instructions live in the control plane; the data never leaves our storage.

Which subscription holds what

Databricks' subscription

MANAGED FOR US — WE NEVER LOG IN

Control plane
Web interface, orchestration, Unity Catalog, saved code and jobs.

Serverless compute
Machines Databricks supplies and bills; nothing to provision.

Our Azure subscription

OUR TENANT, OUR NETWORK, OUR BILL

Classic compute
Cluster VMs started in a resource group we own.

Workspace storage
The ADLS account holding every managed table.

Our own resources
Source databases, files and services the platform reads.

Security: our data stays in our tenant — reviews only cover our side.

Cost: classic VMs land on the Azure bill; serverless is billed by Databricks.

03

Anatomy of the platform

Get familiar with Databricks vocabulary.

account & workspace

compute

namespace

Account and workspace: what lives where

ACCOUNT — ONE PER ORGANISATION

account console

Identity, governance and billing live here

- Users, groups and service principals
- Unity Catalog metastore
- Account-level settings and audit
- Billing and usage across workspaces

Workspace — dev

Notebooks, jobs, dashboards, compute and its own permissions.

Workspace — test

Same shape, separate environment, separate access list.

Workspace — prod

Production pipelines and reporting; tightly restricted.

> Governance is set once at the account and applies to every workspace under it.

Compute is the machine your code runs on

YOU WRITE

Notebook, query or pipeline

Instructions only — code on its own does nothing.

→

COMPUTE RUNS IT

Virtual machines with Spark

Started on demand, sized by you, billed while running.

→

AGAINST

Tables in ADLS Gen2

Data stays in storage; compute is temporary.

Storage and compute are separate. Turning compute off never loses data — it only stops the bill.

Three kinds of compute

All-purpose cluster

Interactive development. You start it, attach notebooks, and several people can share it.

Job cluster

Created for a scheduled job and terminated when it finishes. Cheaper and isolated.

SQL warehouse

Compute tuned for SQL and BI only — powers queries, dashboards and Power BI.

> The workload decides which one you attach it to. Interactive work is not a job, and a job should not run on a shared cluster.

Classic or serverless

CLASSIC — IN OUR SUBSCRIPTION

VMs we can see

Runs in a managed resource group in our Azure subscription.

Inside our network, so private endpoints and firewall rules apply.

Takes minutes to start; VM cost appears on the Azure bill.

SERVERLESS — SUPPLIED BY DATABRICKS

Capacity on tap

Databricks supplies and manages the machines; nothing to provision.

Ready in seconds, which suits short queries and bursty work.

Billed entirely through Databricks, not as Azure VMs.

Either way, the data stays in our ADLS Gen2 account. The choice is about startup time, networking and where the cost lands.

How work is defined and run

FOR PEOPLE

Notebook

A document of cells holding SQL, Python or Scala, run one at a time while you develop.

UNATTENDED

Job

One or more tasks run on a schedule or trigger, with retries, alerting and a run history.

DECLARATIVE

Pipeline

You declare the tables and their sources; Lakeflow works out the order and keeps them current.

Notebooks are where work is written. Jobs and pipelines are how it reaches production.

The three-level namespace

```
prod.claims.policy_fact
```

`catalog`

The top grouping

Usually an environment or a business domain. Governed at the metastore.

`schema`

A group of tables

Also called a database. Where most day-to-day grants are made.

`table`

The data itself

Or a view, a function or a model — all named the same way.

> Permissions can be granted at any level, so how you lay out catalogs and schemas is a security decision.

Volumes: files under the same governance

A volume is a governed folder for files that are not tables — documents, incoming CSVs, model artefacts, images.

It lives inside a schema, so it carries a three-part name and is granted exactly like a table. Before volumes, teams mounted a storage path directly and governance stopped at the door.



Managed or external tables

MANAGED — THE DEFAULT

Unity Catalog owns the files

Databricks chooses the storage location within the metastore root.

Layout and file optimisation are handled automatically.

Dropping the table deletes the underlying data.

EXTERNAL — WHEN SOMETHING ELSE OWNS IT

We own the path

Files sit at a location we nominate, reached via an external location.

Optimisation and cleanup are our responsibility.

Dropping the table leaves the files in place.

Use managed unless another system also writes to those files. This comes back on day two, under storage credentials and external locations.

If you come from SQL Server

server instance	→	**Workspace**	database	→	**Catalog + schema**
query window	→	**Notebook or SQL editor**	SQL Agent job / cron	→	**Job**
the server's CPU and RAM	→	**Compute you start and stop**	logins and GRANT	→	**Entra ID + Unity Catalog**

Close enough to get started. The difference that matters: storage, compute and governance are three separate things here.

04

Where Databricks sits in the Azure estate

It does not replace Azure. It sits inside it and uses the services we already run.

ADLS Gen2 | Entra ID | Key Vault | Monitor | Cost Management

A first-party Azure resource

AZURE SUBSCRIPTION

RESOURCE GROUP — OURS

Where we create the workspace

- The Azure Databricks workspace resource
- Our ADLS Gen2 storage accounts
- Key Vault, Log Analytics, networking

MANAGED RESOURCE GROUP — LOCKED

Created and owned by the service

- Cluster VMs and disks
- The managed virtual network
- Workspace system storage account

> You can see the managed resource group, but you do not deploy into it or change it by hand.

Where the data physically lives

IN OUR ADLS GEN2 ACCOUNT

Every byte of our data

Delta table files, in open Parquet format.

Volume contents — documents, incoming files, artefacts.

Our subscription, our region, our encryption keys.

IN THE DATABRICKS CONTROL PLANE

Metadata about it

Table definitions, permissions and lineage.

Notebook source, job definitions, query history.

No copies of our table data.

> Databricks reads our storage when a query runs. It does not become the store.

Entra ID is the only source of identity

Sign-in

Single sign-on with the normal company account. Conditional access and MFA apply unchanged.

Groups

Synchronised to the account and used for grants, so joiners and leavers are handled once.

Service principals

The non-human identity production jobs run as, rather than a person's account.

Managed identities

Azure-native credentials the platform uses to reach storage, with no secret to rotate.

There is no separate Databricks user directory to keep in step.

Secrets belong in Key Vault

STORED IN

Azure Key Vault

The vault's own access policies, audit trail and rotation apply.

→

EXPOSED AS

A secret scope

A named, permissioned view of the vault inside the workspace.

→

USED AS

```
dbutils.secrets.get( scope, key)
```

Redacted in output, so it cannot leak into logs.

No keys, passwords or connection strings in notebook source or job parameters.

Where the logs go

AZURE MONITOR + LOG ANALYTICS

The estate-wide view

Diagnostic settings stream workspace logs into Log Analytics.

Sign-ins, cluster and job events, workspace configuration changes.

Our existing retention, alerting and KQL queries apply.

UNITY CATALOG SYSTEM TABLES

The platform’s own record

Billable usage, query history, audit and lineage — as tables.

Queried in SQL, so a dashboard can be built over them.

The usual starting point for cost and access questions.

> Configured on day three. For now: two destinations, two purposes.

How consumption reaches the bill

DBU

The Databricks charge

A unit of processing, metered per second. The rate depends on workload type and tier.

+

UNDERLYING COMPUTE

The Azure charge

For classic compute only: the VMs, disks and networking, billed as normal Azure resources.

→

ONE INVOICE

Azure Cost Management

Both lines appear on the Azure bill, sliceable by subscription, resource group and tag.

> Serverless has no separate VM line — the machines are Databricks', so the whole cost arrives as DBUs.

Tag clusters and warehouses from day one, or the bill cannot be attributed to a team.

Networking, in outline

Managed VNet

Classic compute runs inside a VNet Azure creates. It can instead be injected into a VNet we design.

Private endpoints

Keep traffic to storage and to the control plane off the public internet.

IP access lists

Restrict which networks can reach the workspace UI and API at all.

Session 3 covers the design decisions. For today, just know these three levers exist.

The reference architecture

CONTROL PLANE — MANAGED BY DATABRICKS

no customer data

- Workspace UI & notebooks
- Job scheduler
- Cluster manager
- Unity Catalog
- Query history & audit

↕ secure channel — control commands down, metadata and logs up

OUR AZURE SUBSCRIPTION

COMPUTE PLANE — managed VNet

- Job & all-purpose clusters
- SQL warehouses
- Serverless compute

STORAGE — ADLS Gen2

- bronze
- silver
- gold

IDENTITY
Microsoft Entra ID — SSO, groups

INGEST
Data Factory · Event Hubs · IoT Hub

CONSUME
Power BI · Databricks Apps

SUPPORT
Key Vault · Monitor · Cost Management

> Reused across all three days — every later topic hangs off one of these boxes.

05

How work happens, and who does what

From data landing in storage to a report on someone's screen — and who owns each step.

end-to-end flow

platform roles

shared responsibility

From source system to report

STEP 1

Data lands

Source systems write files into ADLS Gen2, or events arrive continuously.

→

STEP 2

It is processed

A job or pipeline runs on compute, cleaning, joining and reshaping it.

→

STEP 3

It becomes a governed table

Registered in Unity Catalog with a three-part name, grants and lineage.

→

STEP 4

It is consumed

Dashboards, Power BI and notebooks read it through a SQL warehouse.

Every term from today sits at one of these four points.

The administrative roles

ACCOUNT LEVEL

Account admin

Identity, workspace creation, account settings and billing. Only this role can open the account console.

GOVERNANCE

Metastore admin

Owns Unity Catalog: catalogs, grants, external locations and storage credentials.

ONE WORKSPACE

Workspace admin

Users, compute policies and settings inside a single workspace — and nothing outside it.

EVERYONE ELSE

Consumers

Engineers, analysts and scientists, with access granted through Entra ID groups.

> The common trap: expecting a workspace admin to fix an account-level problem. They cannot see it.

Shared responsibility

MICROSOFT

The Azure infrastructure

Data centres, hardware, hypervisor.

Storage durability and encryption at rest.

The Entra ID and Key Vault services.

DATABRICKS

The platform

The control plane and its availability.

The runtime, and patching the service.

Unity Catalog and audit as a capability.

US

Everything we configure

Who has access, and to which data.

Network design, secrets and key policy.

Data quality, retention and cost control.

> Almost every incident on a well-run platform originates in the third column.

Into the workspace

Everything from the last two hours, on screen, in the order we introduced it.

01
The sidebar and what each area is

02
A notebook and its attached compute

03
The compute page and a SQL warehouse

04
Catalog Explorer: catalog, schema, table, volume

05
The account console, and the boundary it draws

Where that leaves you

You can describe what the platform is and does, use its terminology accurately, and locate each Databricks component against the Azure services we already run.

The model

One governed copy of the data on open storage, with Delta Lake making it trustworthy.

The vocabulary

Account, workspace, compute, notebook, job, catalog, schema, table, volume.

The estate

Data in our ADLS Gen2, identity in Entra ID, secrets in Key Vault, cost on the Azure bill.

Questions — ten minutes. Session 2 goes into the architecture in depth.

Q&A · 10 MIN

Platform Architecture

How Azure Databricks is built and deployed — what Databricks operates, what we operate, and where the boundaries fall.

two planes | workspace strategy | the landing zone

By the end, you will be able to

01 Explain the control plane and compute plane, and describe what runs in each.

02 Distinguish classic and serverless compute planes and their respective data-residency and networking implications.

03 Compare workspace deployment models and articulate the trade-offs between isolation, cost and operational overhead.

04 Place Databricks components onto our Azure landing zone, including subscription, resource group and managed resource group boundaries.

06

Control plane and compute plane

What Databricks runs, what we run, and where our data travels.

control plane | classic | serverless | managed resource group

The platform splits into two planes

WHO

Users & applications

Browser, BI tools, APIs.

→

Control plane

RUN BY DATABRICKS

- Web application & notebooks
- Job scheduling and cluster management
- Unity Catalog — permissions and lineage
- Saved queries, code and job definitions

↔

Compute plane

WHERE WORK ACTUALLY RUNS

Classic

Clusters on VMs in our own Azure subscription.

Serverless

Databricks supplies the machines; ready in seconds.

↔

OURS

Cloud storage

Every table and file stays in our ADLS account.

Instructions live in the control plane; the data never leaves our storage.

What Databricks runs in the control plane

Web application

The workspace UI, the notebook editor, the SQL editor and the REST API surface.

Job scheduling

Decides a job is due, orders the compute, tracks the run and handles retries.

Cluster management

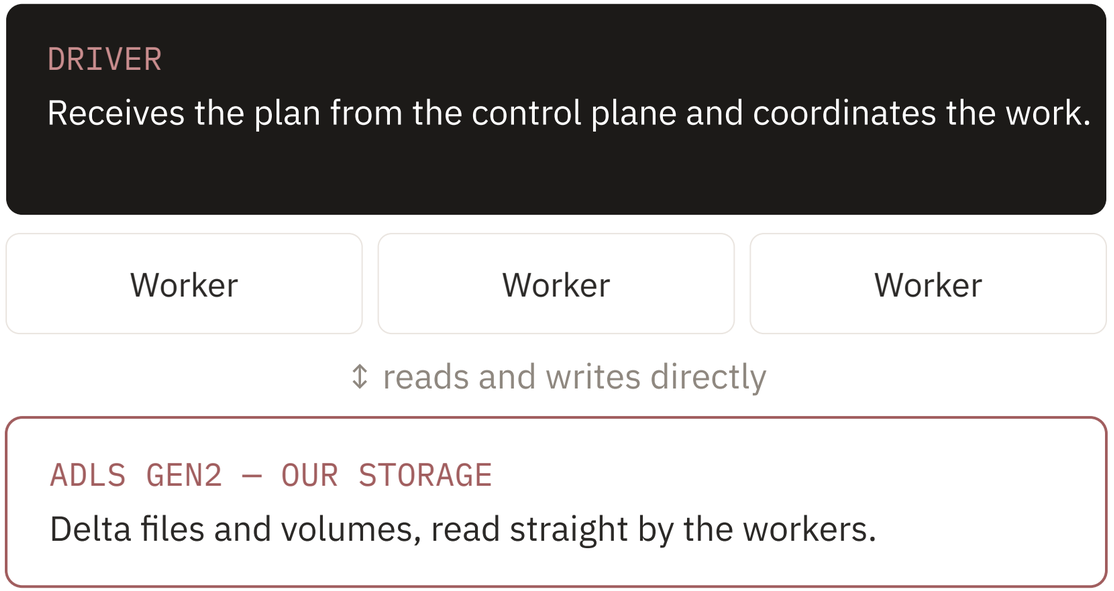
Provisions machines, applies policies, autoscales and terminates them when idle.

Metadata

Table definitions, permissions, lineage, query history, notebook source and job definitions.

All of it runs in a Databricks-operated subscription in our region. We never log into it.

What runs in the compute plane



The compute plane is the only place our data is processed. Clusters and SQL warehouses live here, and so do the jobs and pipelines that use them.

Between the planes only two things move: the plan going down, and status, logs and metadata coming up. The data itself travels between the workers and our storage account, and nowhere else.

Classic: the machines are ours

OUR SUBSCRIPTION · OUR NETWORK

VMs in a managed resource group

Provisioned when the cluster starts, released when it terminates.

Inside a VNet

Managed by default, or injected into a VNet we design and control.

Our network controls apply

Routing, NSGs, private endpoints, firewall egress and ExpressRoute.

Choose it when the network path must be controlled — private endpoints, on-premises sources, strict egress rules.

The cost is a start-up of several minutes, and VM charges on our Azure bill alongside the DBUs.

Serverless: the machines are Databricks’

DATABRICKS-MANAGED SUBSCRIPTION · OUR REGION

Capacity on tap

Seconds to start, nothing to size, patch or terminate.

Isolated per workload

Compute is not shared between customers, and is torn down after use.

Billed as DBUs only

No VM line on the Azure bill — the whole charge comes through Databricks.

Choose it when the work is short, bursty or interactive and start-up time dominates the experience.

The catch is the network: compute sits outside our VNet, so our NSGs and routes do not apply to it.

Residency and networking, compared

	CLASSIC	SERVERLESS
Compute location	Our subscription, our region	Databricks' subscription, our region
Data at rest	Our ADLS Gen2 account	Our ADLS Gen2 account — unchanged
Network control	Our VNet, NSGs, routes and firewall	Network connectivity configurations, not our VNet
Start-up	Minutes	Seconds
Cost lands as	DBUs plus Azure VM charges	DBUs only

Residency is the same in both. The real difference is which network controls apply to the compute.

Where our data does and does not travel

STAYS IN OUR ESTATE

The data itself

Delta files and volume contents in ADLS Gen2.

Rows read by workers while a query runs.

Results returned to the person who asked for them.

GOES TO THE CONTROL PLANE

Metadata about it

Table and column names, permissions, lineage.

Query text and history; cluster and job event logs.

Notebook source and job definitions.

WORTH NAMING EXPLICITLY

Query results are cached in the control plane so the interface can redisplay them, and notebook cell output is saved with the notebook. Both are configurable, and both are covered in session three.

The managed resource group

WHAT AZURE PUTS IN IT

- Cluster VMs, disks and network interfaces
- The managed VNet and its security group
- Workspace system storage — DBFS root, cluster logs, job output

System storage is not where our governed tables live.

WHY IT IS LOCKED

A deny assignment stops anyone — including subscription owners — from changing resources the service depends on. Azure creates it, names it after the workspace, and manages its lifecycle.

Do not deploy into it, rename its resources or delete it.

Do read it for cost attribution, and expect Azure Policy to see it.

> Deleting the managed resource group breaks the workspace, and it cannot be recreated by hand.

07

Workspace strategy and deployment models

How many workspaces, split along which lines, and what each choice costs.

isolation patterns trade-offs discussion

A workspace is a boundary for four things

Security

Compute policies, IP access lists and network configuration are set per workspace.

Isolation

People in one workspace cannot see another's notebooks, jobs or clusters.

Cost attribution

Consumption is reported per workspace, which makes it the natural unit for chargeback.

Blast radius

A bad policy, a runaway cluster or a failed deployment is contained inside one workspace.

Data access is the exception — that is governed at the account by Unity Catalog, not by the workspace.

Split by environment

DEV

Open enough to work

Engineers create their own compute; relaxed policies; sample or masked data.

→

TEST

Shaped like production

Same policies and deployment path, so a release is genuinely rehearsed.

→

PROD

Locked down

No interactive access, restrictive cluster policies, changes only through a pipeline.

Why it works: production is protected from experimentation, and that argument applies everywhere.

What it costs: three of everything to configure, and a promotion path that has to be built.

> Adopt this by default. The other patterns layer on top of it rather than replacing it.

Split by business unit

Actuarial

Own budget, own compute policies, own workspace admin.

Claims

Insulated from the others; its own workload profile and schedules.

Finance

Separate chargeback line, separate access list.

Why it works: clean cost attribution, and one team's sprawl never reaches another's.

What it costs: duplicated configuration, and collaboration becomes a governance exercise.

> Worth it where budgets are genuinely separate. Expensive where teams share pipelines daily.

Split by data sensitivity

GENERAL WORKSPACE

Everyday analytics

Serverless available; broad access list.

Standard egress and network configuration.

Most teams and most workloads live here.

RESTRICTED WORKSPACE

Regulated or personal data

Private connectivity only; public access disabled.

Egress restricted; exfiltration controls enforced.

Short, reviewed access list and stricter policies.

Use it when a control applies to the environment. If the requirement is “these people cannot read that column”, that is a grant, not a workspace.

The hybrid most enterprises land on

PRIMARY AXIS — ENVIRONMENT

Dev

One shared workspace.

Test

One shared workspace.

Production

Split again by business unit, plus one restricted workspace for regulated data.

`prod-actuarial` `prod-claims` `prod-restricted`

> Add a second axis only where you can name the requirement it satisfies. Every extra workspace is a permanent operational cost.

What more workspaces cost you

Operational overhead

Each one needs its own policies, monitoring, access reviews and deployment target — and they drift apart unless deployment is automated.

Cross-workspace sharing

Unity Catalog shares data across workspaces easily. Notebooks, jobs, dashboards and cluster policies do not cross the boundary.

Governance consistency

The more workspaces, the harder it is to demonstrate that a control is applied to all of them — which is what an auditor will ask.

All three are arguments for automating workspace deployment, not for having fewer workspaces than the design needs.

Account, workspaces and the metastore

DATABRICKS ACCOUNT

WORKSPACES — COMPUTE & COLLABORATION

dev | test | prod

↓ all assigned to ↓

METASTORE — DATA

One per region. Holds catalogs, schemas, tables and every grant.

Several workspaces can share one metastore, so a table is governed once and read consistently from all of them — with no copy.

The workspace is the compute and collaboration boundary; the metastore is the data boundary. They are deliberately not the same thing.

Detailed on day two — catalogs, grants, external locations and lineage.

Which pattern fits us?

Four questions. The answers become the input to the landing zone design in the next module.

01

Are our budgets genuinely separate by team?

02

Is there data that needs an environment-level control, not just a grant?

03

How much admin capacity do we actually have?

04

Will workspace deployment be automated from day one?

08

Mapping onto the Azure landing zone

Subscriptions, regions, policy and the shared services the platform depends on.

placement | region | policy & tagging | shared services

Subscription and resource group placement

MANAGEMENT GROUP — DATA PLATFORM POLICY SET

Subscription — dev

Subscription — test

Subscription — prod

RG — WORKSPACE

The workspace resource. One per workspace.

RG — DATA

ADLS Gen2 accounts. Outlive any workspace.

RG — SUPPORTING

Key Vault, Log Analytics, private endpoints.

RG — MANAGED

Created by Azure per workspace. Locked.

Split resource groups by lifecycle, not by product. Storage should survive a workspace being rebuilt.

Region, and what depends on it

A region is the Azure location a workspace physically lives in. Pick it for where the data is legally allowed to sit, then for closeness to the data and the people using it. It is hard to change later.

The metastore is regional

The catalog of our tables belongs to one region. Example: a workspace in West Europe cannot use the catalog in East US.

Features arrive by region

New Databricks features appear in some regions before others. Example: serverless compute may not exist yet in the region you pick.

Continuity follows region

If the Azure region goes down, the workspace goes with it. Example: recovery means a second workspace in another region.

> Keep the workspace, its storage and its data sources in one region. Reading across regions is slower and Azure charges for it.

Policy, tagging and naming

Azure Policy

An Azure-wide rule that says what resources people are allowed to create. Azure checks every new resource against it, including the ones Databricks creates for us.

Example: a rule blocking public IPs stops a cluster from starting, and the error appears in Databricks.

Tagging

Labels of your own choosing attached to a resource, used to tell the finance team whose cost it is.

Example: tag a cluster team=actuarial so its compute cost shows under that team in the Azure bill.

Naming

The agreed pattern for what we call each resource. Databricks copies the workspace name into the resources it creates, and they cannot be renamed later.

Example: adb-actuarial-prod-weu, fixed before the first deployment.

Test these rules with a real cluster start before go-live. A policy denial looks like a Databricks fault, so the cause is easy to misread.

Where we depend on shared services

IDENTITY TEAM

Identity

The tenant, group provisioning, conditional access and service principal creation.

NETWORK TEAM

Networking

VNets and subnets, private endpoints, DNS, firewall rules and ExpressRoute.

OPERATIONS

Logging

The central Log Analytics workspace, retention policy and the SIEM behind it.

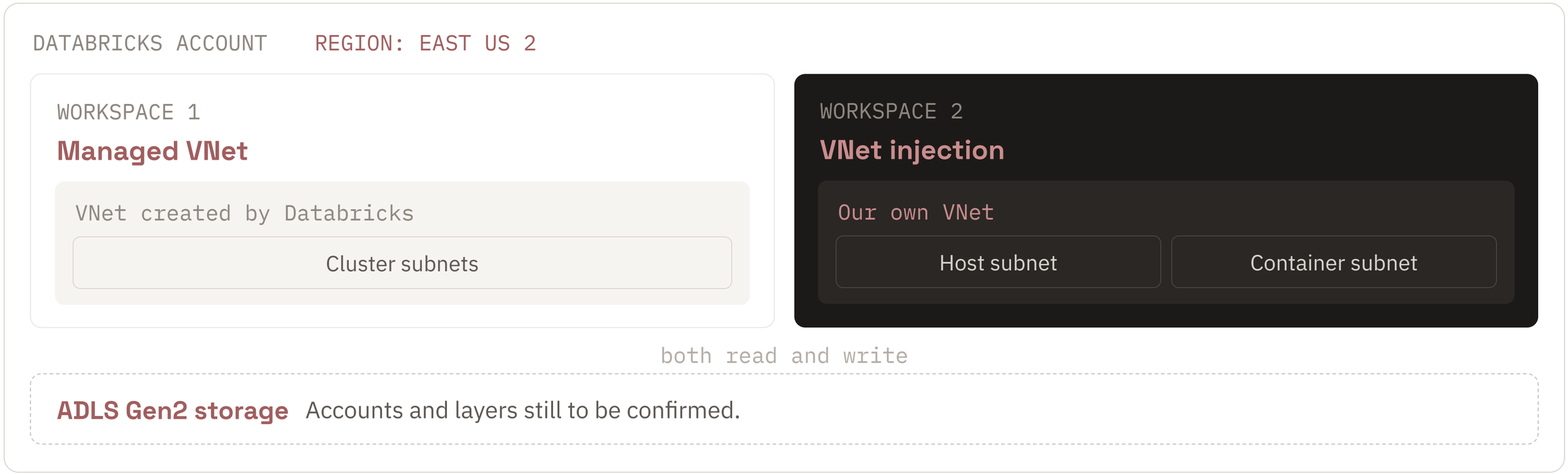
SECURITY

Key management

Key Vault, customer-managed keys and the rotation policy they must follow.

> Each of these is another team's backlog with its own lead time. Networking is usually the long pole — start it first.

Our layout, against the reference diagram



Same account and region, two different network models — a useful side-by-side when we reach the network session.

Where that leaves you

You can describe the platform's architecture in terms of what we control and what Databricks controls, explain the reasoning behind the workspace layout, and position the platform correctly within the wider Azure environment.

The two planes

Instructions and metadata above, data and compute below, in our own subscription.

The workspace layout

Environment first, a second axis only where a named requirement justifies it.

The landing zone

Subscriptions by environment, resource groups by lifecycle, shared services named and owned.

Session 3 goes into the network design — the longest and most technical block of the day.

NEXT · 2H 30M

Network Design and Private Connectivity

The network configuration options for an enterprise deployment, the controls that govern traffic in and out, and the patterns that let the workspace be reached privately.

VNet injection | traffic controls | Private Link | hub and spoke

By the end, you will be able to

01 Compare VNet injection with the Databricks-managed VNet and justify the choice for an enterprise deployment.

02 Describe the role of network security groups, user-defined routes and subnet delegation in a Databricks deployment.

03 Explain front-end, back-end and web authentication private connectivity, and what each protects against.

04 Describe how the deployment integrates with our hub-and-spoke network topology.

Every control is introduced with the risk it mitigates, so the session is useful without configuration-level detail.

09

Networking fundamentals for Databricks

What talks to what, the default posture, and the risk that makes enterprises change it.

the five conversations | default posture | public access

What actually needs to talk to what

01	**Users → control plane**	Browsers, BI tools and API clients reaching the workspace UI and API.
02	**Compute → control plane**	The cluster calling home for its plan, reporting status and shipping logs.
03	**Compute → ADLS Gen2**	The heavy one. Workers reading and writing every byte of our data.
04	**Compute → data sources**	On-premises databases, SaaS APIs and event streams the pipelines read.
05	**Compute → the internet**	Library installs and package repositories — forgotten until a cluster fails to start.

> Every control in this session applies to one of these five arrows. Ask which one, each time.

The default posture

A managed VNet

Azure creates it with two subnets and an NSG carrying the rules the service needs.

A public workspace URL

Reachable from any network on the internet, subject to Entra ID sign-in.

Public endpoints

Compute reaches the control plane and storage over the public internet, encrypted in transit.

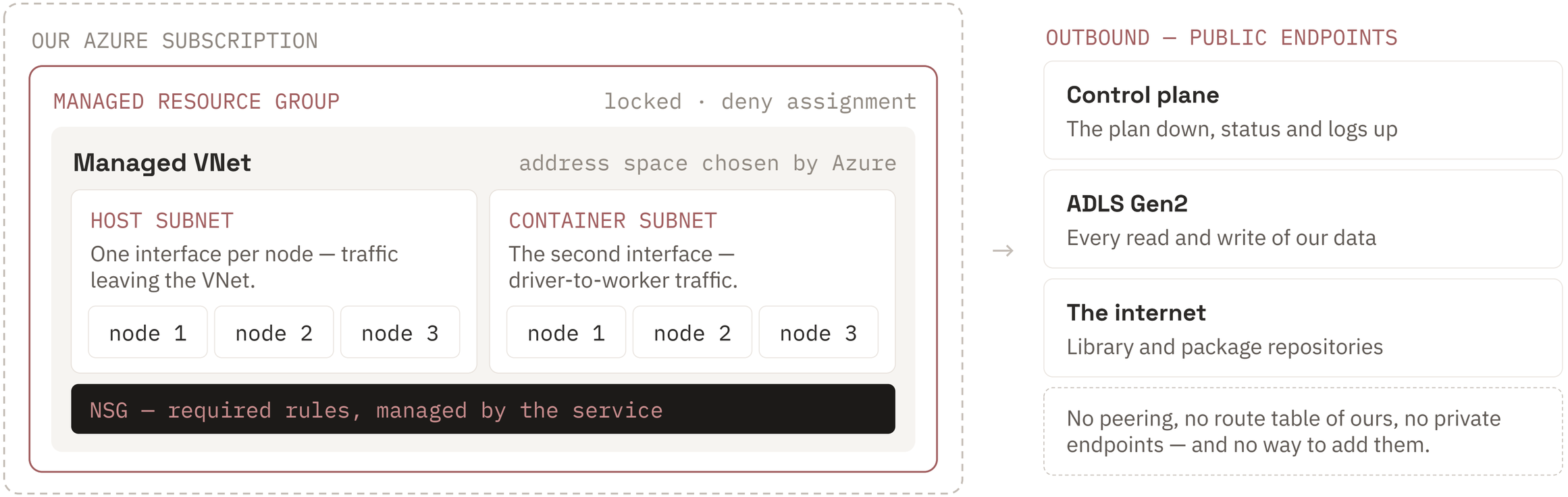
WHY IT IS THE DEFAULT

It works immediately, there is nothing to design, and it is perfectly sound for a proof of concept.

WHY ENTERPRISES CHANGE IT

Everything rests on authentication alone. There is no network-layer control to fall back on, and no path to on-premises sources.

Inside the managed VNet



It sits in our subscription, but Azure owns it. That constraint is the whole argument for VNet injection.

What public network access actually risks

Credential compromise

If someone steals a password or an access token, they can sign in from anywhere in the world. Nothing else is needed.

Exfiltration

If a cluster can reach the whole internet, someone can copy our data out to any outside address. It looks like normal traffic from a normal machine.

Insider misuse

Someone with real access, working from a personal laptop or another country, looks the same as anyone else on the network.

Network rules do not replace logins and permissions. They add a second condition: the credential also has to be used from a network we approve.

10

VNet injection or the managed VNet

Whose virtual network the compute plane runs in — and everything that follows from it.

managed VNet | injection | subnets & CIDR

The managed VNet: a closed box that works

WHAT YOU GET

A VNet created inside the locked managed resource group.

Two subnets, correctly sized and delegated for you.

An NSG already carrying the rules the service requires.

Nothing to design, and no way to misconfigure it.

WHAT YOU CANNOT DO

Choose the address space, or avoid overlap with our own.

Peer it to another VNet, or reach an on-premises source.

Route egress through a firewall or a proxy.

Attach a private endpoint to reach storage privately.

> Fine until a requirement involves reaching something on our own network. Then it is the wrong model.

VNet injection: our network, our rules

DATABRICKS WORKSPACE

Runs the clusters

NIC →

OUR VNET AND SUBNETS

Cluster machines get network interfaces here — ours to design and own

Peering

Connect the spoke to the hub, and through it to the rest of the estate.

Routing and NSGs

Force egress through a firewall and write our own security rules.

Private endpoints

Reach ADLS, Key Vault and the control plane without the public internet.

On-premises

A route to source systems over ExpressRoute or VPN.

The price: the network becomes our design, our review and our responsibility.

The constraint: the VNet and subnets must exist before the workspace, and cannot change without redeploying it.

Two subnets, both delegated

OUR VNET — DATABRICKS SPOKE

HOST SUBNET

Every cluster VM has one network interface here. This is the one that talks to the outside — the Databricks control plane, ADLS storage and any source systems.

Example: a VM here reads a file from ADLS Gen2.

CONTAINER SUBNET

Every cluster VM has a second interface here. This one only talks to the other VMs in the same cluster.

Example: the driver sends work to a worker over this interface.

Azure's docs call these "public" and "private," but neither is exposed to the internet — the names just describe which of the two jobs above the interface does.

Delegation to the Databricks service reserves each subnet — nothing else can deploy into it.

Both the same size, and dedicated to one workspace only.

Both carry the NSG whose required rules the service manages.

CIDR sizing caps your cluster scale

THE RULE Each cluster node takes one address in each subnet. Azure reserves five per subnet. Both subnets are the same size.

/26

About 59 usable addresses — roughly 29 nodes across the entire workspace.

/24

About 251 usable — roughly 125 nodes. A reasonable starting point for a team workspace.

/22

About 1,019 usable — roughly 509 nodes, for a busy shared production workspace.

> Size for peak concurrency, not the average. The failure mode is clusters refusing to start with an address-exhaustion error — usually at month end.

Inside an injected VNet

OUR AZURE SUBSCRIPTION

OUR VNET — THE DATABRICKS SPOKE

address space we choose

Two delegated subnets

created before the workspace

HOST SUBNET · /24

Delegated to Databricks — sized by us.

node 1 | node 2 | node 3

CONTAINER SUBNET · /24

Same size, same delegation.

node 1 | node 2 | node 3

NSG — required rules + ours

ROUTE TABLE — ours

The managed resource group is still there and still locked — it now holds the VM resources, not the network.

↔

WHAT THIS UNLOCKS

Peering to the hub

Firewall, DNS and the gateway become reachable

Forced tunnelling

Our route table sends egress to the firewall

Private endpoints

ADLS, Key Vault and the control plane, privately

On-premises

A path over ExpressRoute or VPN

Compare with the previous diagram: the machines are identical. What differs is who owns the network they sit in.

When injection is required, not preferred

01 Data must be reached over private endpoints.

02 Clusters must reach an on-premises system.

03 Egress must pass through a firewall or proxy.

04 Data exfiltration protection is a requirement.

05 Network flow logs must reach our SIEM.

— In a regulated enterprise, several of these are true. That settles it.

Decide before the workspace is created. Switching afterwards means redeploying it — so this is a day-one conversation with the network team, not a later hardening step.

11

Traffic controls

Security groups, routing, egress control — and what each one is actually protecting against.

NSGs · UDRs · exfiltration · serverless egress

Network security groups

A network security group (NSG) is a list of allow and deny rules attached to a subnet. It decides what traffic can enter and leave.

RULES DATABRICKS ADDS

Needed for the workspace to work at all. We cannot remove them.

Example: the driver VM talking to worker VMs.

Example: a cluster reaching the Databricks control plane.

Example: a cluster reading from workspace storage.

Delete one of these and clusters stop starting — the error rarely mentions the NSG.

RULES WE ADD OURSELVES

Optional, on top of the required rules.

Example: allow traffic to one named vendor API.

Example: deny traffic we never expect to see.

Example: export flow logs so we can review traffic later.

WHAT AN NSG DOES NOT DO

The rules Databricks adds are broad on purpose, so an NSG alone does not answer "what can this cluster reach on the internet?" — that needs a firewall.

Forcing all traffic through a firewall

SOURCE

Cluster subnet

A cluster VM tries to reach the internet.

→

ROUTE TABLE

A rule we add

Redirects all outbound traffic to the firewall instead of straight to the internet.

→

INSPECTION POINT

Firewall in the hub

Checks the request against an allow list of approved sites and logs it.

→

DESTINATION

Approved endpoints only

Anything not on the list is blocked.

The catch: this same path also carries the cluster's own traffic back to Databricks. That traffic must be added to the allow list too, or routed around the firewall.

> Getting that wrong leaves clusters stuck in pending — the most common failure in a hardened deployment.

Data exfiltration protection

THE PROBLEM

Anyone who can run code on a cluster can write the data it is entitled to read anywhere the network allows. Unity Catalog governs which tables they can read, not where the bytes go afterwards.

01

No public IPs

Cluster nodes get no public address, so there is no direct outbound path.

02

Forced tunnelling

A default route sends every outbound connection to the firewall.

03

FQDN allow list

Only named destinations are permitted; everything else is dropped and logged.

04

Private endpoints

Storage traffic stays on the Azure backbone and never touches the internet.

The operational cost is real: every new library source or API must be added to the allow list. Make that request fast, or people will work around it.

Serverless changes the controls

Serverless compute is not in our VNet, so our routes and NSGs do not reach it. The controls exist, but they are different objects configured in a different place.

NETWORK CONNECTIVITY CONFIGURATIONS

How serverless reaches our resources

Private endpoints from serverless compute to our storage.

A stable set of source addresses our firewalls can allow.

Created at the account and attached to workspaces.

SERVERLESS EGRESS CONTROL

Where serverless may reach out to

Restricts which external destinations workloads can call.

Configured in Databricks, not in Azure networking.

The serverless counterpart to the firewall allow list.

> These capabilities move quickly. Check current availability against the requirement before committing a regulated workload to serverless.

Thank you

Day 2 moves into Unity Catalog and governance.